AI Engineering

RAG | LangChain | LangGraph

✔ End-to-End Architecture & Concepts
✔ Components, Workflows & Data Flow
✔ Installation, Libraries & Code Examples

Theory Guide — 2024 / 2025 Edition

Table of Contents

Chapter 1 — RAG: Retrieval-Augmented Generation

1.1 What Is RAG?

Retrieval-Augmented Generation (RAG) is an AI architecture that enhances a Large Language Model (LLM) by connecting it to an external knowledge base at inference time. Instead of relying solely on the parametric knowledge baked into the model's weights during training, RAG retrieves relevant documents on-the-fly and injects them into the prompt — giving the model grounded, up-to-date, and domain-specific context to generate its answer.

> ■ **KEY INSIGHT**
>
> Key insight: LLMs have a knowledge cutoff and a finite context window. RAG solves both problems by fetching only the relevant pieces of a large corpus and presenting them inline.

1.2 Why RAG?

Benefit	Explanation
Hallucination reduction	Model answers are grounded in real retrieved documents
Fresh knowledge	Corpus can be updated without retraining the LLM
Domain specificity	Private docs, internal wikis, legal/medical corpora
Cost efficiency	Cheaper than full fine-tuning; minimal GPU required
Traceability	Every answer can cite its source documents
Long-context bypass	Only relevant chunks sent — no 1M-token context needed

1.3 RAG Architecture — End-to-End

RAG has two main phases: **Indexing** (offline) and **Retrieval + Generation** (online).

Phase 1 — Indexing Pipeline (Offline)

Raw documents are transformed into searchable vector representations and stored.

Step	What Happens
1. Document Loading	PDFs, HTML, DOCX, databases, APIs are loaded into memory
2. Text Splitting	Documents chunked into overlapping segments (e.g. 512 tokens, 50 overlap)
3. Embedding	Each chunk converted to a dense vector via an embedding model
4. Vector Storage	Vectors + metadata stored in a vector database (FAISS, Pinecone, Chroma)

Phase 2 — Query Pipeline (Online / Real-Time)

Step	What Happens
1. Query Embedding	User question embedded using the SAME embedding model
2. Similarity Search	Top-k nearest vectors retrieved from the store (cosine / dot-product)
3. Context Assembly	Retrieved chunks concatenated with the user query into a prompt
4. LLM Generation	Prompt sent to LLM; model generates an answer grounded in context
5. Post-Processing	Optional: re-ranking, citation extraction, output parsing

1.4 Components Deep-Dive

Document Loaders

Connectors that ingest raw data from various sources into a unified Document object.

- **PyPDFLoader** — loads PDF files page by page
- **WebBaseLoader** — scrapes HTML pages (uses BeautifulSoup)
- **CSVLoader / JSONLoader** — structured data sources
- **DirectoryLoader** — batch-loads an entire folder
- **S3FileLoader / GCSFileLoader** — cloud storage loaders

Text Splitters

LLMs have context-window limits; splitting keeps chunks meaningful and under the token budget. The two main parameters are **chunk_size** (max tokens per chunk) and **chunk_overlap** (tokens shared between consecutive chunks to preserve context across boundaries).

- **RecursiveCharacterTextSplitter** — preferred; splits on \n\n, \n, ' ', '' progressively
- **CharacterTextSplitter** — splits on a single separator
- **TokenTextSplitter** — splits by actual token count (tiktoken)
- **MarkdownHeaderTextSplitter** — structure-aware for Markdown
- **HTMLHeaderTextSplitter** — respects HTML heading hierarchy

Embedding Models

Maps text to a fixed-size dense vector in semantic space. Similar texts yield similar vectors.

Model	Notes
OpenAI text-embedding-3-small	1536-dim, fast, cost-effective
OpenAI text-embedding-3-large	3072-dim, higher accuracy
HuggingFace BAAI/bge-m3	Multilingual, open-source, 1024-dim

Cohere embed-v3	Strong for retrieval tasks
Ollama (local)	Fully offline — nomic-embed-text

Vector Stores

Store	Characteristics
FAISS	In-memory, open-source, great for prototyping (Facebook AI)
Chroma	Open-source, persistent, easy local setup
Pinecone	Managed cloud, production-scale, serverless option
Weaviate	Open-source + cloud, supports hybrid search (BM25 + vectors)
Qdrant	Rust-based, fast, filterable, open-source
pgvector	PostgreSQL extension — add vectors to existing Postgres DB

Retrievers

- **Similarity Search (k-NN)** — cosine similarity, returns top-k chunks
- **MMR (Maximal Marginal Relevance)** — balances relevance vs diversity
- **Multi-Query Retriever** — LLM generates multiple query rewrites, merges results
- **Contextual Compression Retriever** — LLM distills each chunk before returning
- **BM25 / Hybrid Retriever** — combines keyword (sparse) + semantic (dense) search
- **Self-Query Retriever** — LLM auto-generates metadata filters
- **Parent-Document Retriever** — retrieves small chunks, returns parent chunk for context

1.5 RAG Variants

Variant	Description
Naive RAG	Basic retrieval + generation — good starting point
Advanced RAG	Pre-retrieval (query rewrite) + post-retrieval (re-rank) stages
Modular RAG	Plug-in architecture — swap any component independently
Graph RAG	Knowledge graph as the retrieval backbone (relationships matter)
Agentic RAG	LLM agent decides WHEN and HOW to retrieve (tool calling)
Corrective RAG	Evaluates retrieval quality; web-searches if context is weak
Self-RAG	Model generates reflection tokens to critique its own output

1.6 RAG — Installation & Code

Installation

```
# Core RAG stack<br/>pip install langchain langchain-community langchain-openai<br/>p
ip install chromadb faiss-cpu tiktoken<br/>pip install pypdf unstructured<br/><br/>#
Optional: HuggingFace embeddings (local/free)<br/>pip install sentence-transformers<b
r/><br/># Optional: Pinecone cloud vector store<br/>pip install pinecone-client
```

End-to-End RAG — Minimal Example

```
from langchain_community.document_loaders import PyPDFLoader<br/>from langchain.text_
splitter import RecursiveCharacterTextSplitter<br/>from langchain_openai import OpenA
IEmbeddings, ChatOpenAI<br/>from langchain_community.vectorstores import Chroma<br/>f
rom langchain.chains import RetrievalQA<br/><br/># 1. Load document<br/>loader = PyPD
FLoader("my_document.pdf")<br/>docs = loader.load()<br/><br/># 2. Split into chunks<b
r/>splitter = RecursiveCharacterTextSplitter(<br/>    chunk_size=1000, chunk_overlap=
150<br/>)<br/>chunks = splitter.split_documents(docs)<br/><br/># 3. Embed + store in
Chroma<br/>embeddings = OpenAIEmbeddings()   # or HuggingFaceEmbeddings()<br/>vectord
b = Chroma.from_documents(chunks, embeddings, persist_directory="./chroma_db")<br/><b
r/># 4. Retriever<br/>retriever = vectordb.as_retriever(search_type="mmr", search_kwa
rgs={"k": 5})<br/><br/># 5. QA Chain<br/>llm = ChatOpenAI(model_name='gpt-4o-mini', t
emperature=0)<br/>qa_chain = RetrievalQA.from_chain_type(<br/>    llm=llm,<br/>    re
triever=retriever,<br/>    chain_type="stuff",<br/>    return_source_documents=True<b
r/>)<br/><br/># 6. Query<br/>result = qa_chain.invoke({"query": "What is the refund p
olicy?"})<br/>print(result["result"])<br/>for doc in result["source_documents"]:<br/>
    print(doc.metadata)
```

LCEL RAG Chain

```
from langchain_core.prompts import ChatPromptTemplate<br/>from langchain_core.runnabl
es import RunnablePassthrough<br/>from langchain_core.output_parsers import StrOutput
Parser<br/><br/>prompt = ChatPromptTemplate.from_template(<br/>    "Answer based only
 on: {context}\n\nQuestion: {question}"<br/>)<br/><br/>def format_docs(docs):<br/>
 return "\n\n".join(d.page_content for d in docs)<br/><br/>rag_chain = (<br/>    {"co
ntext": retriever | format_docs, "question": RunnablePassthrough()}<br/>    | prompt<
br/>    | llm<br/>    | StrOutputParser()<br/>)<br/><br/>print(rag_chain.invoke("What
 is the main topic of the document?"))
```

Chapter 2 — LangChain: The LLM Application Framework

2.1 Core Philosophy

LangChain is an open-source framework that provides standardised abstractions and composable building blocks for developing LLM-powered applications. Its philosophy is **composability**: every component (LLM, prompt, retriever, tool, memory) implements the same Runnable interface, so they chain together seamlessly using the pipe (**|**) operator via LCEL.

■ ARCHITECTURE PHILOSOPHY

LangChain separates concerns into distinct layers: the core abstractions (langchain-core), the integrations (langchain-community + first-party packages), and the orchestration chains (langchain). This lets you swap providers without rewriting business logic.

2.2 Architecture Layers

Package / Service	Role
langchain-core	Base abstractions: Runnable, BaseMessage, BasePromptTemplate, Document. Zero heavy deps.
langchain-community	600+ third-party integrations: loaders, stores, LLMs, tools. Community-maintained.
langchain	High-level chains, agents, and cognitive architectures built on top of core.
langchain-openai	First-party OpenAI integration (ChatOpenAI, OpenAIEmbeddings).
langchain-anthropic	First-party Anthropic integration (ChatAnthropic).
langchain-google-*	Google Vertex, Gemini, Google Search integrations.
LangSmith	Observability platform — tracing, debugging, evaluation, prompt management.
LangServe	Deploy LangChain chains as REST APIs (FastAPI under the hood).

2.3 Key Components

2.3.1 Chat Models & LLMs

The central component. LangChain wraps every provider behind a unified interface. Chat models take a list of **BaseMessage** objects and return a message.

```
from langchain_openai import ChatOpenAI<br/>from langchain_anthropic import ChatAnthr
opic<br/>from langchain_community.llms import Ollama<br/><br/># OpenAI<br/>llm = Chat
```

```
OpenAI(model='gpt-4o', temperature=0.7, streaming=True)<br/><br/># Anthropic Claude<br/>llm = ChatAnthropic(model='claude-3-5-sonnet-20241022')<br/><br/># Local via Ollama<br/>llm = Ollama(model='llama3.2')
```

2.3.2 Prompt Templates

Structured, reusable prompt definitions that accept variables and output formatted messages.

```
from langchain_core.prompts import (<br/>    ChatPromptTemplate, SystemMessagePromptTemplate,<br/>    HumanMessagePromptTemplate, FewShotChatMessagePromptTemplate<br/>)<br/><br/># Basic chat prompt<br/>prompt = ChatPromptTemplate.from_messages([<br/>    ("system", "You are a {role} assistant."),<br/>    ("human",  "Help me with: {task}"),<br/>])<br/><br/># Render<br/>msgs = prompt.format_messages(role="Python expert", task="decorators")
```

2.3.3 Output Parsers

Transform raw LLM output into structured Python objects.

Parser	Output
StrOutputParser	Returns plain string — simplest, always works
JsonOutputParser	Parses JSON string into Python dict
PydanticOutputParser	Validates and returns a Pydantic model instance
CommaSeparatedListOutputParser	Splits output on commas → Python list
XMLOutputParser	Parses XML-tagged output
DatetimeOutputParser	Parses date strings into datetime objects

2.3.4 Memory

Persist conversation history across turns so the model remembers past exchanges.

- **ConversationBufferMemory** — stores full message history (grows unbounded)
- **ConversationBufferWindowMemory** — keeps last k messages
- **ConversationSummaryMemory** — LLM summarises older messages to save tokens
- **ConversationSummaryBufferMemory** — hybrid: recent messages + summary of older
- **VectorStoreRetrieverMemory** — retrieves most relevant past turns
- **Entity Memory** — tracks facts about named entities mentioned in conversation

2.3.5 Tools & Tool Calling

Tools let LLMs call external functions (web search, calculator, database, APIs). Modern chat models support **function/tool calling** natively — the model returns a structured JSON payload specifying which tool to call and with what arguments.

```
from langchain_core.tools import tool<br/><br/>@tool<br/>def get_weather(city: str) -> str:<br/>    """Return current weather for a given city."""<br/>    return f'Sunny, 22C in {city}'   # replace with real API<br/><br/># Bind tools to the model<br/>llm_with_tools = llm.bind_tools([get_weather])<br/>response = llm_with_tools.invoke('What is the weather in Berlin?')
```

2.3.6 Agents

Agents use an LLM as a reasoning engine to decide which tools to call and in what order. The dominant pattern is **ReAct** (Reason + Act): the model interleaves Thought, Action, and Observation steps until it can produce a final answer.

```
from langchain.agents import create_react_agent, AgentExecutor<br/>from langchain import hub<br/><br/># Pull pre-built ReAct prompt from LangChain Hub<br/>prompt = hub.pull("hwchase17/react")<br/><br/>agent = create_react_agent(llm, tools=[get_weather], prompt=prompt)<br/>executor = AgentExecutor(agent=agent, tools=[get_weather], verbose=True)<br/><br/>result = executor.invoke({"input": "Is it warmer in Berlin or Munich?"})
```

2.4 LCEL — LangChain Expression Language

LCEL is the declarative composition layer in LangChain. Every component implements **Runnable**, providing a consistent API: **.invoke()**, **.stream()**, **.batch()**, and **.ainvoke()** (async). Chains are built with the **|** pipe operator.

Method	Purpose
invoke(input)	Synchronous single-call
stream(input)	Yields tokens as they are generated
batch([i1, i2, ...])	Parallel invocation over a list
ainvoke / astream	Async equivalents (asyncio-compatible)
with_config({})	Override callbacks, tags, run_name per invocation
with_fallbacks([...])	Fallback to other runnables on error
with_retry()	Auto-retry on transient errors

```
# LCEL composition examples<br/><br/># Simple chain<br/>chain = prompt | llm | StrOutputParser()<br/><br/># Parallel execution with RunnableParallel<br/>from langchain_core.runnables import RunnableParallel, RunnableLambda<br/><br/>parallel = RunnableParallel({<br/>    "summary": prompt_summary | llm | StrOutputParser(),<br/>    "keywords": prompt_keywords | llm | StrOutputParser(),<br/>})<br/><br/># Branching with RunnableBranch<br/>from langchain_core.runnables import RunnableBranch<br/>branch = RunnableBranch(<br/>    (lambda x: 'code' in x['topic'], code_chain),<br/>    (lambda x: 'math' in x['topic'], math_chain),<br/>    default_chain   # fallback<br/>)
```

2.5 LangChain — Installation & Code

Installation

```
# Minimal<br/>pip install langchain langchain-core<br/><br/># With OpenAI<br/>pip install langchain-openai<br/><br/># With Anthropic<br/>pip install langchain-anthropic<br/><br/># Full community integrations<br/>pip install langchain-community<br/><br/># Observability (LangSmith)<br/>pip install langsmith<br/><br/># Environment variables<br/>export OPENAI_API_KEY='sk-...'<br/>export LANGCHAIN_API_KEY='ls__...'<br/>export LANGCHAIN_TRACING_V2='true'
```

Conversation Chain with Memory

```
from langchain.chains import ConversationChain<br/>from langchain.memory import ConversationBufferWindowMemory<br/>from langchain_openai import ChatOpenAI<br/><br/>memory = ConversationBufferWindowMemory(k=5, return_messages=True)<br/>llm    = ChatOpenAI(model='gpt-4o-mini')<br/><br/>conv = ConversationChain(llm=llm, memory=memory, verbos
```

```
e=True)<br/><br/>conv.predict(input='Hi! My name is Alex.')<br/>conv.predict(input='W
hat is my name?')   # remembers Alex
```

Structured Output with Pydantic

```
from pydantic import BaseModel, Field<br/>from langchain_openai import ChatOpenAI<br/
>from langchain_core.prompts import ChatPromptTemplate<br/><br/>class Movie(BaseModel
):<br/>    title:  str = Field(description="Movie title")<br/>    year:   int = Field
(description="Release year")<br/>    genre:  str = Field(description="Primary genre")
<br/><br/>llm = ChatOpenAI(model='gpt-4o-mini').with_structured_output(Movie)<br/><br
/>prompt = ChatPromptTemplate.from_messages([<br/>    ("human", "Recommend a movie ab
out {topic}"),<br/>])<br/><br/>chain = prompt | llm<br/>movie = chain.invoke({"topic"
: "artificial intelligence"})<br/>print(movie.title, movie.year, movie.genre)
```

Chapter 3 — LangGraph: Stateful Agent Graphs

3.1 What Is LangGraph?

LangGraph is a library built on top of LangChain that models LLM workflows as **directed graphs**. Unlike linear chains, graphs can loop, branch, and maintain shared state — making them ideal for complex, multi-step agent systems, human-in-the-loop workflows, and multi-agent collaboration. LangGraph is production-grade and powers many enterprise AI agents.

> ■ MENTAL MODEL
>
> Mental model: think of LangGraph as a state machine where NODES are Python functions (or LLM calls) and EDGES are the transitions between them. A shared TypedDict 'State' flows through every node and accumulates results.

3.2 Core Concepts

Concept	Description
StateGraph	The graph object — container for nodes, edges, and state schema
State	TypedDict shared across all nodes; annotated with reducers
Node	Python function: receives State, returns partial State update
Edge	Directed connection between two nodes
Conditional Edge	Edge whose next node is decided at runtime by a routing function
START / END	Special sentinel nodes marking entry and exit of the graph
Checkpointer	Persists state snapshots for resumability and time-travel
Interrupt	Pause graph execution to wait for human input
Command	Return value from a node that can route AND update state simultaneously

3.3 Graph Primitives

Defining State

```
from typing import TypedDict, Annotated, List<br/>from langgraph.graph.message import
 add_messages<br/>from langchain_core.messages import BaseMessage<br/><br/>class Agen
tState(TypedDict):<br/>    # add_messages reducer APPENDS rather than overwriting<br/
>    messages: Annotated[List[BaseMessage], add_messages]<br/>    context:  str
  # overwrite on each update<br/>    step_count: int
```

Defining Nodes

```
from langchain_openai import ChatOpenAI<br/><br/>llm = ChatOpenAI(model='gpt-4o')<br/><br/>def call_llm(state: AgentState) -> dict:<br/>    '''Node: call the LLM with the current message history.'''<br/>    response = llm.invoke(state['messages'])<br/>    return {'messages': [response]}   # add_messages will append<br/><br/>def tool_node(state: AgentState) -> dict:<br/>    '''Node: execute the tool called by the LLM.'''<br/>    last_msg = state['messages'][-1]<br/>    results  = execute_tools(last_msg.tool_calls)<br/>    return {'messages': results}
```

Building the Graph

```
from langgraph.graph import StateGraph, START, END<br/><br/>graph = StateGraph(AgentState)<br/><br/># Add nodes<br/>graph.add_node('agent', call_llm)<br/>graph.add_node('tools', tool_node)<br/><br/># Edges<br/>graph.add_edge(START, 'agent')      # always start at agent<br/><br/># Conditional edge — router function<br/>def should_continue(state: AgentState) -> str:<br/>    last = state['messages'][-1]<br/>    if last.tool_calls:               # model wants a tool<br/>        return 'tools'<br/>    return END                        # model is done<br/><br/>graph.add_conditional_edges('agent', should_continue, ['tools', END])<br/>graph.add_edge('tools', 'agent')    # loop back<br/><br/>app = graph.compile()
```

3.4 State Management

State is the single source of truth in a LangGraph application. Each node receives the full State dict and returns a **partial update** — only the keys it changes. LangGraph merges updates using **reducer functions** annotated on each field.

Reducer	Behaviour
Default (overwrite)	Last write wins — simple scalar values
add_messages	Appends new messages to list (built-in for chat history)
operator.add	Appends any list (import operator; Annotated[list, operator.add])
Custom reducer	Any function f(current_value, update_value) -> new_value

3.5 Control Flow & Edges

Edge Types

Pattern	Meaning
add_edge(a, b)	Always go from node a to node b
add_conditional_edges(a, fn, map)	fn(state) returns a key; map routes to next node
add_edge(START, first_node)	Entry point of the graph
add_edge(any_node, END)	Terminal — execution stops
SEND API	Fan-out: dispatch same node with different state slices in parallel

Checkpointing & Human-in-the-Loop

```
from langgraph.checkpoint.memory import MemorySaver<br/><br/>memory = MemorySaver()  # in-memory; use SqliteSaver / PostgresSaver in prod<br/>app = graph.compile(checkpo
```

```
inter=memory, interrupt_before=['tools'])<br/><br/># Run until the interrupt<br/>conf
ig  = {"configurable": {"thread_id": "session-42"}}<br/>snapshot = app.invoke({"messa
ges": [HumanMessage("Delete all my files")]}, config)<br/><br/># Inspect what the mod
el wants to do<br/>pending = snapshot["messages"][-1].tool_calls<br/>print(pending)
 # human reviews here<br/><br/># Approve: resume from snapshot<br/>final = app.invoke
(None, config)   # None = resume, don't re-run from scratch
```

3.6 Multi-Agent Patterns

LangGraph supports building systems where multiple agents coordinate. Common patterns:

Supervisor Pattern

A supervisor agent routes user requests to specialised worker agents. The supervisor is a conditional-edge router; each worker is a node in the same graph.

```
members = ['researcher', 'coder', 'writer']<br/><br/>def supervisor_node(state):<br/>
    # LLM decides which worker should act next<br/>    decision = supervisor_llm.invo
ke(state['messages'])<br/>    return {'next': decision.next_worker}<br/><br/>graph.ad
d_conditional_edges(<br/>    'supervisor',<br/>    lambda s: s['next'],<br/>    {m: m
 for m in members} | {"FINISH": END}<br/>)
```

Subgraph Pattern

Encapsulate complex logic in a subgraph and use it as a node in a parent graph.

```
# Compile inner graph<br/>research_agent = research_graph.compile()<br/><br/># Use as
 a node in the parent graph<br/>parent_graph.add_node('research', research_agent)
```

3.7 LangGraph — Installation & Code

Installation

```
pip install langgraph<br/><br/># With persistence backends<br/>pip install langgraph-
checkpoint-sqlite    # SQLite<br/>pip install langgraph-checkpoint-postgres  # Postgr
eSQL<br/><br/># LangGraph Platform (hosted execution)<br/>pip install langgraph-sdk
```

Complete ReAct Agent with Tools

```
from typing import Annotated<br/>from langchain_openai import ChatOpenAI<br/>from lan
gchain_core.tools import tool<br/>from langchain_core.messages import HumanMessage<br
/>from langgraph.graph import StateGraph, START, END<br/>from langgraph.graph.message
 import add_messages<br/>from langgraph.prebuilt import ToolNode, tools_condition<br/
>from typing import TypedDict, List<br/>from langchain_core.messages import BaseMessa
ge<br/><br/># --- Tools ---<br/>@tool<br/>def multiply(a: int, b: int) -> int:<br/>
  """Multiply two integers."""<br/>    return a * b<br/><br/>@tool<br/>def search(que
ry: str) -> str:<br/>    """Search the web (stub)."""<br/>    return f'Results for: {
query}'<br/><br/>tools = [multiply, search]<br/><br/># --- State ---<br/>class State(
TypedDict):<br/>    messages: Annotated[List[BaseMessage], add_messages]<br/><br/># -
-- Model ---<br/>llm = ChatOpenAI(model='gpt-4o').bind_tools(tools)<br/><br/># --- No
des ---<br/>def call_model(state: State):<br/>    return {'messages': [llm.invoke(sta
te['messages'])]}<br/><br/>tool_node = ToolNode(tools)<br/><br/># --- Graph ---<br/>g
 = StateGraph(State)<br/>g.add_node('model', call_model)<br/>g.add_node('tools', tool
_node)<br/><br/>g.add_edge(START, 'model')<br/>g.add_conditional_edges('model', tools
_condition)  # auto routes<br/>g.add_edge('tools', 'model')<br/><br/>app = g.compile(
)<br/><br/># --- Run ---<br/>for event in app.stream(<br/>    {"messages": [HumanMess
age("What is 7 × 6? Then search for that result.")]},<br/>    stream_mode="values"<br
/>):<br/>    event['messages'][-1].pretty_print()
```

Chapter 4 — RAG + LangChain + LangGraph Combined

In production systems these three technologies are layered: LangChain provides the component primitives, RAG provides grounded knowledge retrieval, and LangGraph orchestrates the overall agent loop with state persistence and conditional routing.

Layer	Responsibility
Layer 3 — Orchestration	LangGraph: agent loop, routing, state, checkpointing, HITL
Layer 2 — Reasoning	LangChain: LLMs, prompts, tools, output parsers, LCEL chains
Layer 1 — Knowledge	RAG: document loading, chunking, embedding, vector retrieval

Agentic RAG with LangGraph

```
# Agentic RAG: agent decides whether to retrieve or answer directly<br/><br/>from lan
gchain_core.tools import tool<br/>from langchain_community.vectorstores import Chroma
<br/>from langchain_openai import OpenAIEmbeddings<br/><br/>vectordb   = Chroma(persi
st_directory='./db', embedding_function=OpenAIEmbeddings())<br/>retriever  = vectordb
.as_retriever(search_kwargs={'k': 4})<br/><br/>@tool<br/>def retrieve_docs(query: str
) -> str:<br/>    """Retrieve relevant documents for a given query."""<br/>    docs =
 retriever.invoke(query)<br/>    return "\n\n".join(d.page_content for d in docs)<br/
><br/># Now use retrieve_docs as a tool in the LangGraph ReAct agent above<br/>tools
= [retrieve_docs, search, multiply]<br/>llm_with_tools = ChatOpenAI(model='gpt-4o').b
ind_tools(tools)<br/><br/># The agent will call retrieve_docs when it needs knowledge
,<br/># and answer directly from context otherwise — pure agentic RAG
```

■ CORRECTIVE RAG LOOP

Corrective RAG flow with LangGraph: (1) retrieve → (2) grade documents → (3) if poor quality → web_search → (4) generate answer → (5) hallucination check → (6) if hallucinated → regenerate. This loop is naturally expressed as a LangGraph graph.

Chapter 5 — Quick Reference Cards

All-in-One pip Installs

```
# Complete AI engineering stack<br/>pip install \<br/>    langchain langchain-core la
ngchain-community \<br/>    langchain-openai langchain-anthropic \<br/>    langgraph
langgraph-checkpoint-sqlite \<br/>    chromadb faiss-cpu pinecone-client \<br/>    py
pdf unstructured \<br/>    tiktoken sentence-transformers \<br/>    langsmith
```

Key Class Cheat-Sheet

Class / Object	Package — Purpose
ChatOpenAI	langchain_openai — OpenAI chat model wrapper
ChatPromptTemplate	langchain_core.prompts — multi-message prompt builder
StrOutputParser	langchain_core.output_parsers — text extraction
PydanticOutputParser	langchain_core.output_parsers — structured output
RecursiveCharacterTextSplitter	langchain.text_splitter — smart chunking
OpenAIEmbeddings	langchain_openai — text-to-vector conversion
Chroma / FAISS	langchain_community.vectorstores — vector DBs
RetrievalQA	langchain.chains — classic RAG chain
ConversationChain	langchain.chains — chain with memory
AgentExecutor	langchain.agents — runs ReAct loop
StateGraph	langgraph.graph — directed stateful graph
add_messages	langgraph.graph.message — list reducer
ToolNode	langgraph.prebuilt — auto executes tool calls
tools_condition	langgraph.prebuilt — routes to tools or END
MemorySaver	langgraph.checkpoint.memory — in-memory checkpointer
interrupt	langgraph.types — pause for human approval
Command	langgraph.types — combined route + state update

Embedding Model Comparison

Model	Provider	Dim	Notes

text-embedding-3-small	OpenAI	1536	Fast, cheap, great general purpose
text-embedding-3-large	OpenAI	3072	Highest accuracy in OpenAI family
BAAI/bge-m3	HuggingFace	1024	Multilingual, free, strong retrieval
embed-english-v3.0	Cohere	1024	Excellent for search/RAG tasks
nomic-embed-text	Ollama/local	768	Fully offline, good quality

LangGraph vs LangChain Agents

Dimension	LangChain Chains/Agents	LangGraph
Use case	Simple Q&A; / RAG	Complex loops, state, HITL
State	Implicit / in memory dict	Explicit TypedDict with reducers
Control flow	Linear or simple branch	Full graph (loops, fan-out, fan-in)
Persistence	External / manual	Built-in checkpointing
Human loop	Custom callbacks	Native interrupt / resume
Multi-agent	Sequential chains only	Supervisor, subgraph patterns
Streaming	Token streaming only	Node-level event streaming

Glossary

Term	Definition
Embedding	Dense vector representation of text capturing semantic meaning

Chunk	Segment of a document used as the unit of retrieval
k-NN search	Find the k nearest vectors by cosine / dot-product similarity
MMR	Maximal Marginal Relevance — diversified top-k retrieval
LCEL	LangChain Expression Language — pipe-based chain composition
Runnable	LangChain base protocol; supports invoke/stream/batch/ainvoke
ReAct	Reason + Act — interleaved thought/action/observation agent pattern
HITL	Human-in-the-Loop — pausing agent for human review/approval
Checkpointer	Snapshot of graph state enabling resume and time-travel
Reducer	Function specifying how node state updates are merged
Tool calling	Structured JSON output from LLM specifying a function + args to run
RAG	Retrieval-Augmented Generation — ground LLM in external knowledge
LLM	Large Language Model (GPT-4, Claude, Gemini, Llama, etc.)
Vector DB	Database optimised for high-dimensional vector similarity search
Tokenisation	Splitting text into sub-word units (tokens) for model processing

End of Document

RAG · LangChain · LangGraph — Complete Theory Reference

Generated for educational use · 2024–2025